Year: 2026–2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Class: T1 **Date of Submission: 14/10/2026**

A Mini Project Report on

ProFix: Home Services Marketplace

A Two-Sided Real-Time Service Booking and Provider Management Platform Delivered Through an Agile and DevOps Pipeline

Submitted in partial fulfilment of the requirements of the course

Agile Software Development & DevOps Lab

Semester V (NEP) · Academic Year 2026–2027 · Class T1

Submitted by

Name	Roll No.
Viraj Bhiwandkar	2401014
Tanmay Ahuja	2401003
Aryan Bansode	2401009

Under the guidance of

Dr. Sanober Shaikh

Department of Artificial Intelligence & Data Science

Thadomal Shahani Engineering College

2026–2027

CERTIFICATE

This is to certify that the mini project entitled **“ProFix: Home Services Marketplace: A Two-Sided Real-Time Service Booking and Provider Management Platform Delivered Through an Agile and DevOps Pipeline”** is a bona fide work of **Viraj Bhiwandkar (2401014)**, **Tanmay Ahuja (2401003)** and **Aryan Bansode (2401009)**, submitted in partial fulfilment of the requirements of the course **Agile Software Development & DevOps Lab**, Semester V, at **Thadomal Shahani Engineering College** during the academic year 2026–2027.

_____ _____

Project Guide **Head of Department**

Dr. Sanober Shaikh Department of AI & Data Science

Date: _____ Place: Mumbai

ABSTRACT

Finding reliable, verified home maintenance professionals (plumbers, electricians, carpenters, housekeepers) traditionally relies on word-of-mouth recommendations, unverified paper contacts, or opaque local agencies. Homeowners face uncertain availability, lack upfront pricing transparency, and have no direct channel to coordinate arrival schedules. Conversely, independent service providers lack digital tools to advertise expertise, manage appointment calendars, or negotiate rates. The ProFix Home Services Marketplace addresses this by providing a unified, full-stack digital platform where homeowners can discover verified providers, book schedule-aware time slots with real-time conflict prevention, negotiate rates through instant WebSocket-powered messaging, and complete secure payments via Razorpay.

The application serves as the vehicle for synthesising all seven experiments of the Agile Software Development & DevOps Lab curriculum into an enterprise-grade product lifecycle. Work was planned in Atlassian Jira as three core epics and seven user stories (totalling 21 story points) organised into an iterative Scrum sprint (PF Sprint 1), with all backlog items driven from To Do to Done. Source code was version-controlled in a shared GitHub repository (Viraj7-hub/ASD-D_MPR) under strict feature-branching conventions (feature/frontend-ui, feature/backend-api, feature/devops-ci-cd) integrated through Pull Requests #1 and #2. The multi-tier architecture—comprising a React 19/Vite 8 SPA served via Nginx, an Express 5 REST and Socket.io backend, and a relational MySQL 8.4 database—is containerised with production Dockerfiles and orchestrated using Docker Compose across an isolated bridge network with automated container health probes. A declarative Jenkins CI/CD pipeline running on macOS automatically polls GitHub, verifies development toolchains, executes code linting and backend syntax checks, packages artifacts, and deploys the running stack to Docker with automated zero-touch HTTP and database smoke checks. Finally, Nagios Core provides continuous monitoring across the frontend portal, backend REST endpoints, MySQL database readiness, and host disk storage.

The delivered platform incorporates 4 major service categories, multi-role session guards, real-time Socket.io chat with active user presence tracking, and dual-script deployment verification. The automated deployment pipeline successfully validates all container healthchecks within 180 seconds, ensuring zero manual downtime between source commit and live service monitoring. This report documents the end-to-end architecture, Agile backlog tracking, containerisation specifications, CI/CD pipeline

declarations, monitoring matrices, and verification outputs.

Keywords: Agile, Scrum, Jira, Git, GitHub, Docker, Docker Compose, Jenkins, CI/CD, Nagios Core, React 19, Express 5, MySQL 8.4, Socket.io, Razorpay.

CONTENTS

Chapter	Title
1	Introduction
2	Requirements and Agile Planning (Experiment 6)
3	System Design
4	Application Implementation
5	DevOps Implementation (Experiments 1–7)
6	Testing
7	Results and Discussion
8	Conclusion and Future Scope
	References
A	Appendix: Running the Project

1. Introduction

1.1 Background

Household maintenance services—such as plumbing, electrical work, carpentry, and housekeeping—represent an essential everyday requirement for residential communities. In conventional setups, finding and scheduling technicians relies on informal directories, unvetted paper registers, or local middlemen. This leads to common operational issues: homeowners cannot verify technician credentials or compare experience, appointment scheduling suffers from double-booking due to lack of real-time slot tracking, pricing is opaque and subject to arbitrary haggling, and there is no record of job completion or payment proofs.

The Agile Software Development & DevOps Lab curriculum introduces modern engineering disciplines across seven distinct experiments: version control, container packaging, multi-container orchestration, continuous integration, automated deployment, Agile project tracking, and infrastructure monitoring. Performing each experiment on isolated, toy examples obscures the collaborative synergy required in commercial software delivery. This mini project synthesises all seven laboratory experiments into a comprehensive, production-grade web application—ProFix: Home Services Marketplace—demonstrating how code moves seamlessly from backlog planning to monitored deployment.

1.2 Problem Statement

To design, implement, containerise, deploy, and monitor a secure, full-stack home services marketplace platform that connects homeowners with verified service professionals with real-time calendar scheduling, live chat negotiation, and online payment processing; and to deliver this platform through a fully automated Agile and DevOps pipeline wherein all requirements are tracked in Jira, version-controlled in Git/GitHub, built and verified through Jenkins CI, orchestrated via Docker Compose, and monitored by Nagios Core.

1.3 Objectives

1. **User Discovery & Service Browsing**: Develop a responsive web interface allowing homeowners to explore categorized home services (Plumbing, Electrical, Carpentry, Housekeeping) and view verified provider profiles with experience and ratings.

2. **Dynamic Slot-Aware Booking**: Implement an interactive calendar booking system that checks backend availability in real time (/available_slots) to prevent double-booking.

3. **Real-Time Communication**: Build a bi-directional WebSocket messaging system using Socket.io that allows clients and providers to negotiate quotes and track online/offline presence.

4. **Secure Transactions & Identity**: Implement session-based authentication with bcrypt password hashing, role-based access control guards, Brevo email OTP password resets, and Razorpay HMAC-SHA256 signature verification.

5. **Agile Lifecycle Management (Jira)**: Organize work according to Scrum principles using Atlassian Jira, structuring requirements into Epics, user stories, estimations (story points), sprint boards, and velocity tracking.

6. **Multi-Branch Version Control (Git/GitHub)**: Manage all codebase assets, Docker configs, and CI/CD pipelines in a GitHub repository (Viraj7-hub/ASD-D_MPR) enforcing feature-branch isolation and Pull Request peer review.

7. **Container Orchestration (Docker & Docker Compose)**: Package frontend (React/Vite on Nginx) and backend (Node.js/Express) into hardened Docker containers and coordinate them with a MySQL 8.4 database service over a private bridge network with custom healthchecks.

8. **Automated CI/CD Pipeline & Monitoring (Jenkins & Nagios)**: Construct a declarative Jenkins pipeline (Jenkinsfile) that polls GitHub, validates toolchains, runs ESLint and syntax checks, packages artifacts, deploys to Docker, and verifies system health via smoke tests and Nagios Core probes.

1.4 Scope

The system delivers a functional home services marketplace operating across four service domains (Plumbing, Electrical, Carpentry, Housekeeping) in a metropolitan testing zone. It supports two primary actor roles: Homeowners (Users) and Service Providers. Administrative user deletion and multi-city geographic mapping are reserved for future releases. The deployment target is a macOS host executing Docker Desktop and Jenkins LTS, mirroring standard cloud virtual machine (EC2/Compute Engine) deployment topologies.

1.5 Mapping of Lab Experiments to the Project

Exp	Lab topic	Application in the ProFix Marketplace System	Section
1	Version control with Git	Shared GitHub repository (Viraj7-hub/ASD-D_MPR), feature branches (feature/frontend-ui, feature/devops-ci-cd), pull requests #1 and #2, three contributors	5.1
2	Containerisation with Docker	Multi-stage Dockerfile for React/Vite/Nginx frontend; non-root user (USER node) Dockerfile for Express backend	5.2
3	Docker Compose	Multi-service stack (compose.yaml): db (MySQL 8.4), backend (Express), frontend (Nginx) on bridge network with health checks	5.3
4	Jenkins CI	Declarative Jenkinsfile: Git checkout, toolchain verification, parallel npm ci, backend syntax check, ESLint static analysis, Vite build	5.4
5	Automated deployment with Jenkins	Branch-conditional deployment (main), secure credential binding (profix-compose-env), zero-touch docker compose up -d, HTTP and MySQL smoke tests	5.5

Table 1.1: Lab experiments and where each is applied

2. Requirements and Agile Planning

2.1 User Roles

Role	Goal	Main screens
Homeowner (User)	Discover verified professionals, schedule appointments without booking conflicts, chat in real time, and pay securely.	Landing Page (/), Sign Up (/signup), User Dashboard (/User_dashboard), Service Directories (/Service_*), Booking Wizard (/booking), My Requests (/my_requests), Real-Time Chat (/chat).
Service Provider	Showcase skills and experience, receive incoming booking requests, accept/decline jobs, negotiate pricing via chat, and track job history.	Provider Registration (/signup), Provider Dashboard (/Provider_dashboard), Booking Management, Chat Inbox (/chat).
DevOps / Quality Team	Plan tasks, automate validation and packaging, deploy containerised releases safely, and monitor live services.	Jira Board & Backlog, GitHub Pull Requests, Jenkins Pipeline Console (:8080), Docker Desktop, Nagios Monitoring (:8088).

Table 2.1: User roles

2.2 Functional Requirements

- **Authentication & Authorization:** Secure registration and login for Users and Providers with bcrypt (10 rounds) password hashing; server-side session cookies with httpOnly and sameSite attributes; role-based route guards (ProtectedRoute).
- **Self-Service Password Reset:** 3-step password recovery flow (/verifyemail, /otp, /newpassword) generating 4-digit numeric OTPs with 10-minute expiry dispatched via Brevo SMTP.
- **Categorized Service Discovery:** Public directory exploring Plumbing, Electrical, Carpentry, and Housekeeping providers displaying experience, city, and direct booking buttons.
- **Dynamic Slot Reservation:** Interactive date selection with month navigation disabling past dates; real-time slot conflict querying via /available_slots to grey out booked hours; multi-step job detail collection with image attachment support.
- **Real-Time WebSocket Messaging:** Socket.io bi-directional chat system mapping user emails to active socket IDs; message history persistence in MySQL Messages

table; live user presence indicator (online/offline green/grey status).

- **Payment Processing:** Integrated Razorpay checkout order generation (/payment/create-order) with server-side HMAC-SHA256 signature verification (crypto.timingSafeEqual) preventing replay and double-payment.
- **Booking State Machine:** Provider dashboard enabling instant Accept/Decline status transitions on incoming customer bookings (PUT /booking_status).
- **Observability Endpoints:** Read-only session verification probe (GET /session-check) reporting unauthenticated 401 status for pipeline smoke testing and Nagios health probes.

2.3 Non-Functional Requirements

- **Reliability:** Every container features healthchecks (MySQL ping test, Node fetch test, Nginx HTTP fetch) with restart policies (unless-stopped). Pipeline enforces pre-flight smoke tests before releasing builds.
- **Portability:** Entire system spins up on any machine running Docker using a single command: docker compose up -d --build.
- **Maintainability:** Infrastructure as Code (IaC) principles: Dockerfiles, compose.yaml, Jenkinsfile, and database migrations are maintained in version control alongside application source.
- **Security:** Backend runs under non-root privilege (USER node, UID 1000); secrets injected through Jenkins credential bindings and environment variables (.env.example template kept in Git); timing-safe signature comparison prevents timing attacks.
- **Observability:** Health status of frontend UI, backend API, database connectivity, and host storage monitored continuously by Nagios Core.

Table 2.4: Distribution of work across the team (as assigned in Jira)

3. System Design

3.1 Architecture

The ProFix Home Services Marketplace is a modular, three-tier containerised web application integrated with external cloud providers and monitored by an external DevOps supervision plane. The web browser executes the React 19 single-page application served via Nginx on published port 5173. The Express 5 backend runs on published port 8009, handling RESTful endpoints, session storage, and Socket.io full-duplex WebSocket connections. The MySQL 8.4 database runs on internal port 3306 on the private bridge network. External integrations include Razorpay for online transactions and Brevo SMTP for OTP password recovery. Continuous integration and monitoring are executed by Jenkins LTS (:8080) and Nagios Core 4.5 (:8088).

Figure 3.1: System and delivery architecture

3.2 Technology Stack

Layer	Technology	Reason for choice
Frontend	React 19, Vite 8, Lucide React, Socket.io Client	Fast reactive UI, zero-latency component re-rendering, lightning Vite HMR and production bundle optimization.
Web server	Nginx 1.25 (Alpine)	Extremely small container footprint (~40 MB), low memory overhead, rapid static SPA delivery.
Backend	Node.js 22, Express 5	Non-blocking asynchronous I/O, native support for JSON payloads, extensive middleware ecosystem.
Real-time	Socket.io	Full-duplex WebSocket communication with graceful fallback, event-driven user presence tracking and chat broadcasting.
Database	MySQL 8.4 Enterprise Relational DB	Strong ACID transactional guarantees, robust foreign-key schema integrity, reliable container initialization.
Authentication	bcrypt (10 rounds), express-session	Salted cryptographically secure password hashing; server-side secure cookie tracking without client token vulnerabilities.
External services	Razorpay SDK, Brevo SMTP	Reliable payment order creation with HMAC signature verification; reliable email delivery

Layer	Technology	Reason for choice
		for OTP verification.
Containers	Docker, Docker Compose	Consistent multi-container environment replication across macOS and Linux servers; isolated bridge networking.
CI/CD	Jenkins LTS (Declarative Pipeline)	Industry-standard automation server, native pipeline-as-code (Jenkinsfile), credential management, automated test reporting.
Monitoring	Nagios Core 4.5	External, proactive service checks validating web responses, API availability, database connectivity, and host disk capacity.

Table 3.1: Technology stack

3.3 Database Design

The relational schema RESUME_PROJECT1 contains five normalized tables enforcing entity relationships between users, providers, bookings, messages, and payments: User_data (Email PK, UserName, Password, phoneNumber, Address), Service_Provider_data (Email PK, UserName, Password, phoneNumber, serviceType, Experience, City), Bookings (id PK, userEmail FK, providerEmail FK, serviceType, bookingDate, timeSlot, description, status, createdAt), Messages (id PK, senderEmail FK, receiverEmail FK, message, timestamp), and Payments (id PK, orderId UK, paymentId, userEmail FK, providerEmail FK, serviceType, amount, currency, status, createdAt).

Figure 3.2: Entity-relationship diagram

3.4 Backend Operations

Operation	Protocol	Endpoint	Purpose
Root & session check	GET	/session-check	Reports session validity and user role; returns 401 when unauthenticated
Registration	POST	/signup	Registers new Homeowner or Service Provider with bcrypt-hashed passwords
Authentication	POST	/login	Validates credentials against role tables and establishes session cookie
Password reset	POST/PU	/verifyemail, /otp,	Generates 4-digit OTP via Brevo SMTP; validates within 10-minute

Operation	Protocol	Endpoint	Purpose
	T	/newpassword	expiry window
Service discovery	GET	/Service_:type	Returns registered providers filtered by category (Plumbing, Electrical, etc.)
Slot availability	GET	/available_slots	Returns booked time slots for a specific provider on a selected date
Create booking	POST	/booking	Creates appointment record initialized with Pending status
Booking transition	PUT	/booking_status	Allows provider to transition status to Accepted, Declined, or Completed
Real-time messaging	WS	Event: chat_message	Broadcasts text messages instantly to shared room; writes to Messages table
Payment order	POST	/payment/create-order	Generates Razorpay transaction order with collision checks
Payment verification	POST	/payment/verify	Verifies Razorpay HMAC-SHA256 signature and updates payment status

Table 3.2: Backend operations

3.5 Business Rules

Situation	Enforced System Behaviour	Error / HTTP Response
Provider Already Booked	Backend checks Bookings for matching providerEmail, bookingDate, and timeSlot. If found, slot is omitted.	Frontend greys out slot; backend rejects duplicate submission with conflict error.
Past Date Selection	Homeowner selects date prior to current calendar day.	Calendar widget disables clicking; backend rejects bookingDate < CURDATE().
Unauthenticated Access	Client navigates to protected page without active cookie.	Route guard triggers /session-check; redirects client to /login.
Self-Payment Attempt	Payer email in payment payload matches provider email.	Transaction aborted; backend returns 400 Bad Request: Self-payment

Situation	Enforced System Behaviour	Error / HTTP Response
		disallowed.
Duplicate Payment Order	User attempts payment on booking that already carries a paid order.	Backend checks Payments table; blocks redundant order creation.
Unverified Password Reset	Direct PUT to /newpassword without prior OTP validation.	Rejected; backend enforces req.session.isOtpVerified === true.

Table 3.3: Allocation rules enforced by the backend

3.6 Security and Configuration Design

- **Non-Root Execution:** The backend container runs under the unprivileged node user (UID 1000) defined in backend/Dockerfile rather than root.
- **Credential Segregation:** Secrets are kept out of Git. The template .env.example is committed, while production database passwords and API tokens are injected via Jenkins withCredentials bindings.
- **Timing-Safe Cryptography:** Razorpay signature verification utilizes crypto.timingSafeEqual() to guard against side-channel timing attacks.
- **Cookie Security:** Session cookies enforce httpOnly: true (preventing XSS access) and sameSite: 'lax'.
- **Database Isolation:** MySQL port 3306 is isolated within the internal bridge network resume_project_default and is not exposed to the external host.

4. Application Implementation

The application consists of a React 19 single-page frontend served via Nginx and an Express 5 REST and WebSocket backend. This chapter walks through the running application using the team's screenshots.

4.1 Dashboard and Landing Interface

Figure 4.1 shows the public landing page opened at localhost:5173. It presents the core brand identity, a value proposition showcase ("Why Choose Us"), verified professional trust badges, and direct navigation hooks for user registration and login.

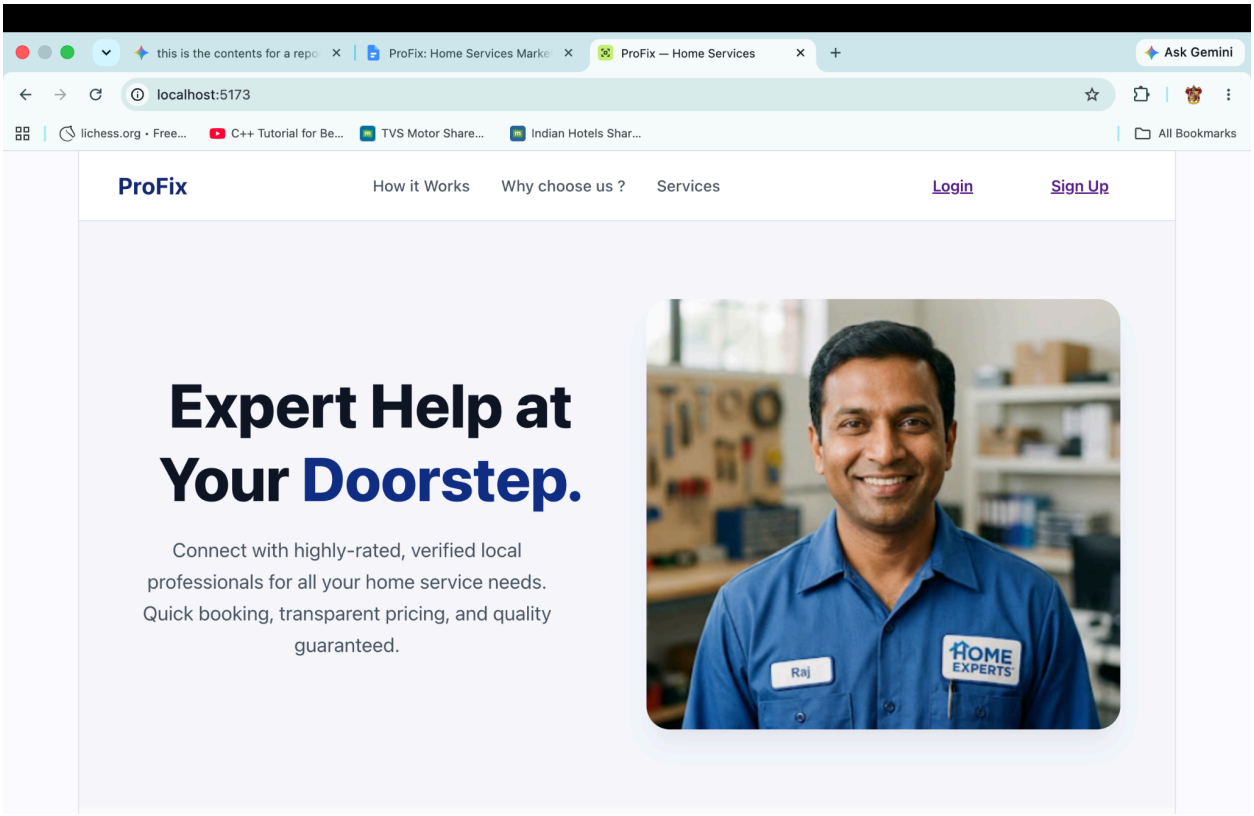


Figure 4.1: Public Landing Page and Value Proposition Showcase

Figure 4.2 illustrates the authenticated Homeowner Dashboard. It greets the active user by name ("Namaste, Viraj Bhiwandkar!"), provides quick-access category tiles for Plumbing, Housekeeping, Carpentry, and Electrical work, and offers a navigation drawer for tracking service requests and me

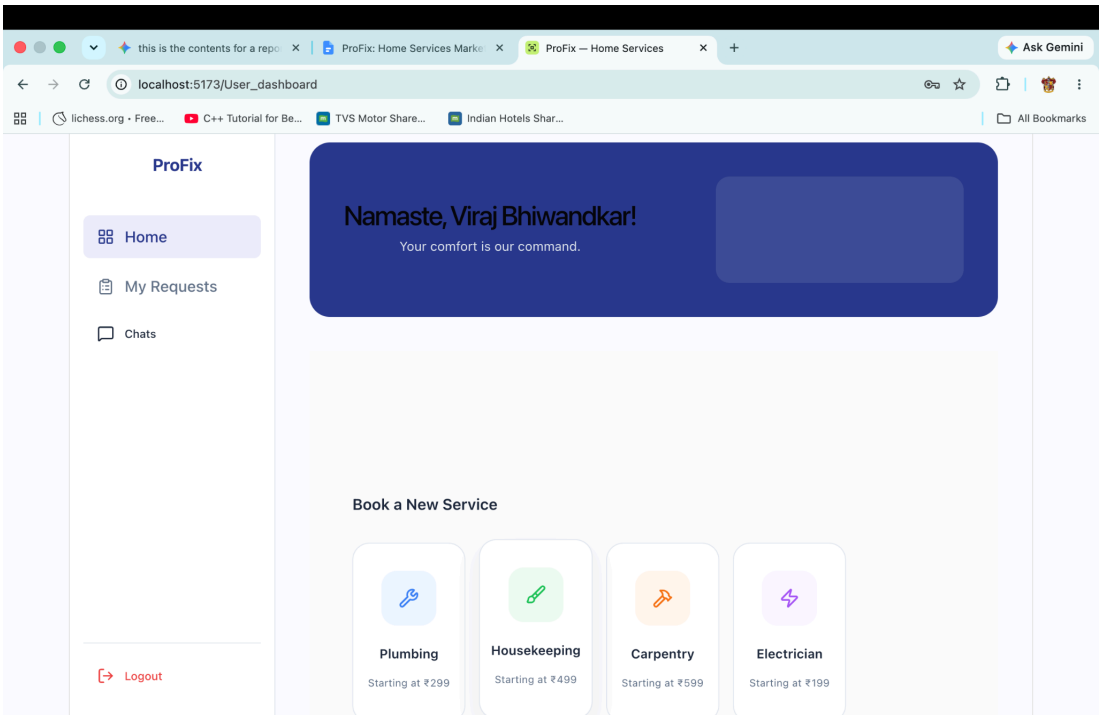
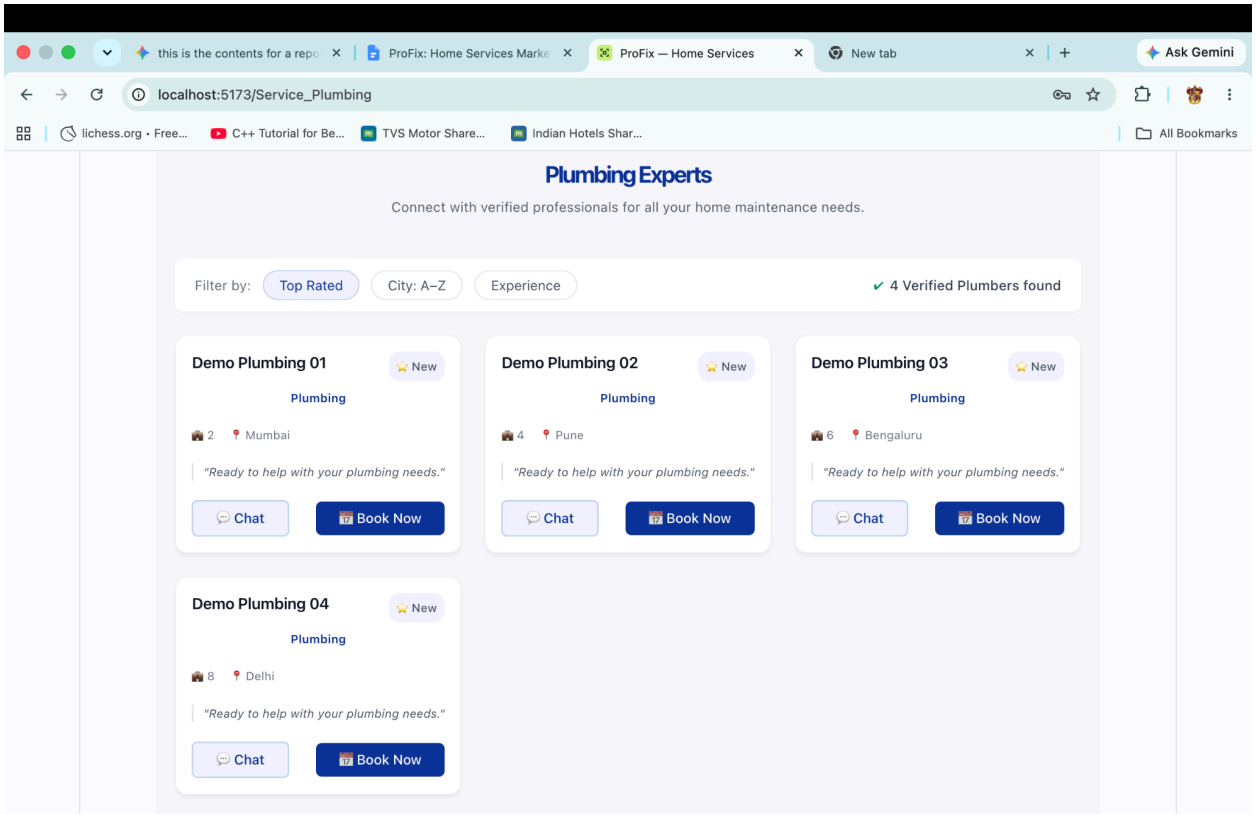


Figure 4.2: Homeowner Dashboard Interface

4.2 Key Features and Service Discovery

Figure 4.3 shows the Plumbing Experts directory (/Service_Plumbing). Active verified technicians are listed with their city, years of experience, and action buttons. Homeowners can click “Book Now” to launch the scheduling wizard or “Chat” to open the real-time negotiation channel.

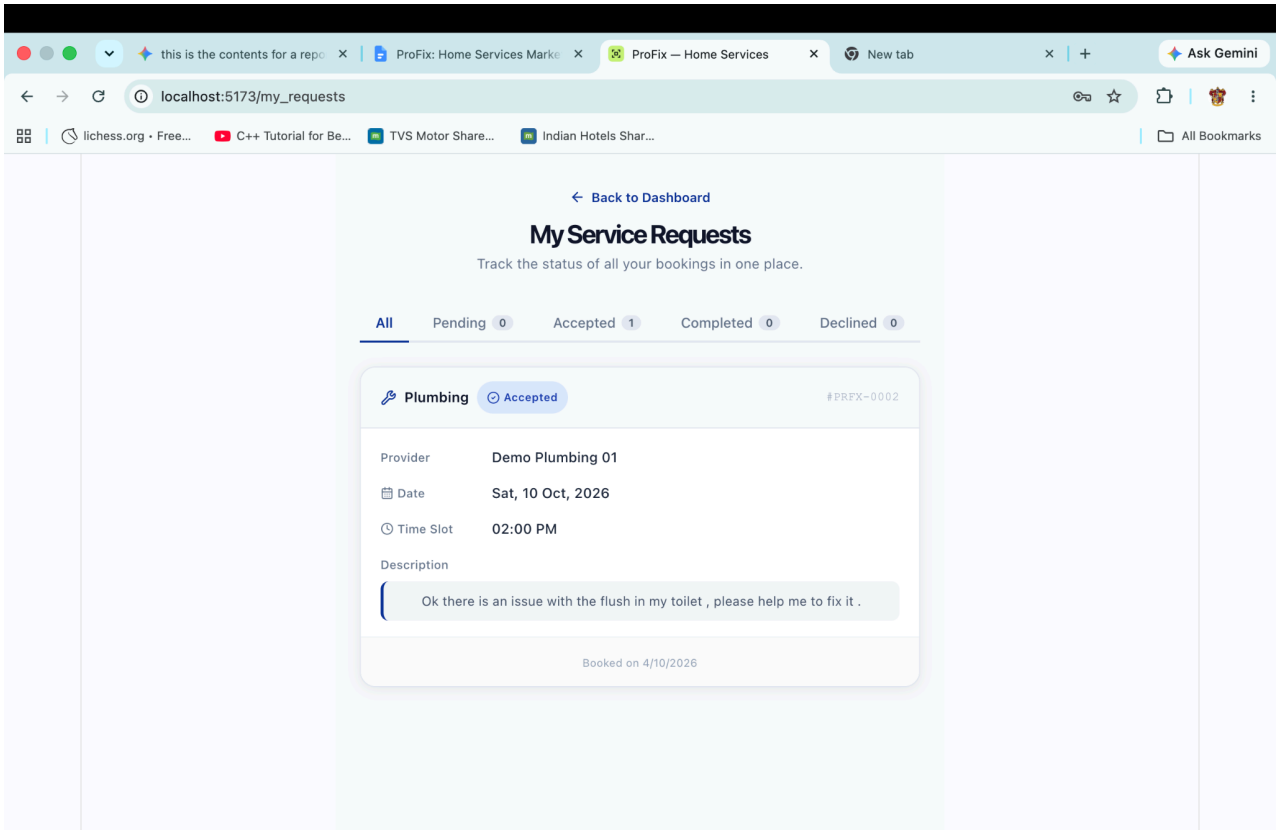
Figure 4.3: Service Discovery Directory for Plumbing Experts



4.3 Booking Engine and Calendar Slot Selection

Figure 4.4 displays the My Service Requests tracking portal (/my_requests). Bookings reflect their live status (Accepted, Pending, Completed, Declined), appointment date, reserved time slot, and diagnostic notes.

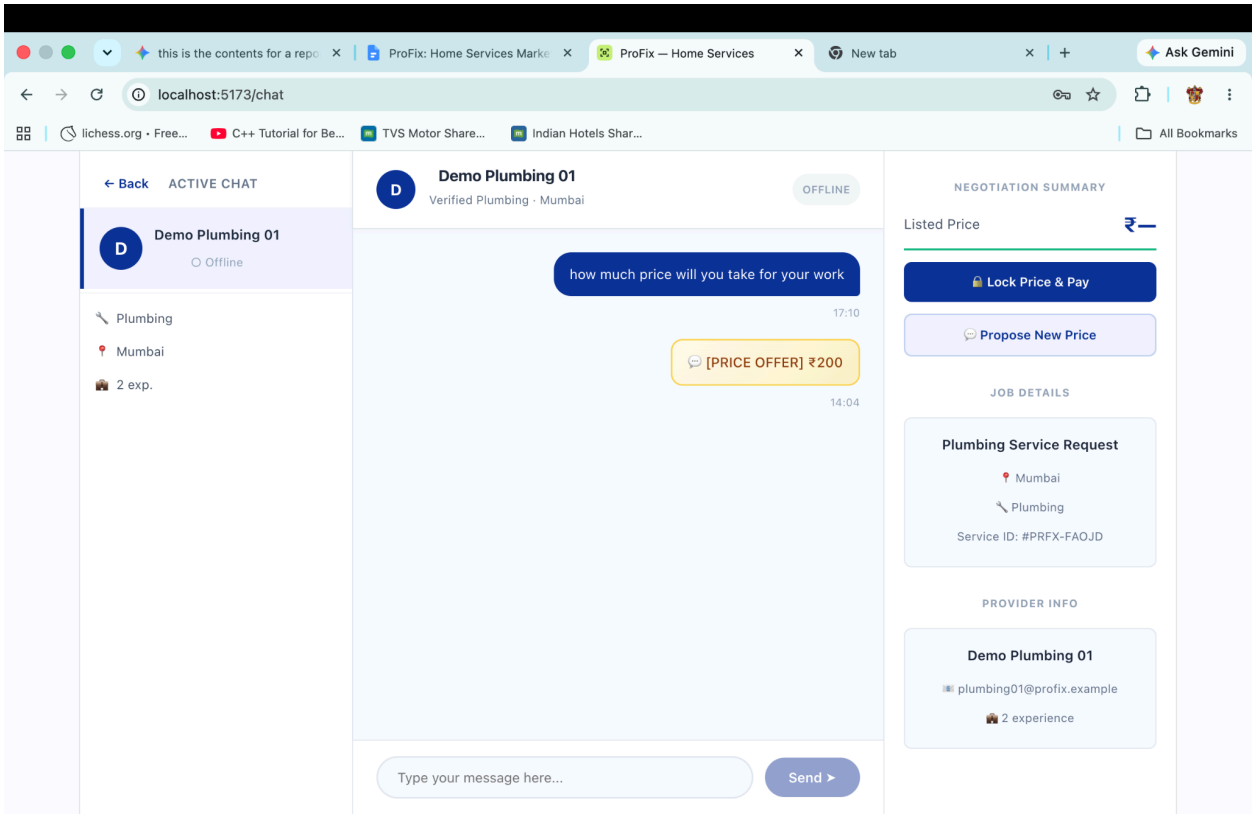
Figure 4.4: My Service Requests Tracking Interface



4.4 Real-Time Chat and Payment Gateway Integration

Figure 4.5 captures the real-time WebSocket chat and negotiation interface (/chat). Homeowners and providers exchange messages, track online presence, submit structured price offers (“[PRICE OFFER] ₹200”), and proceed directly to payment.

Figure 4.5: Real-



Time Chat & Price Negotiation Interface

4.5 Backend Logic and Session Validation

The Express 5 application runs inside Node.js 22. It validates incoming booking payloads, ensures date and time slot integrity, and exposes /session-check to verify session state across Docker healthchecks, Jenkins smoke tests, and Nagios active probes.

5. DevOps Implementation

This chapter describes how each lab experiment was applied to the ProFix Marketplace. The delivery pipeline ties together Jira planning, Git branching, Pull Request peer review, Jenkins CI/CD automation, Docker orchestration, and Nagios Core infrastructure monitoring.

5.1 Version Control with Git and GitHub (Experiment 1)

All application source code, Docker assets, configuration blueprints, CI/CD pipelines, and verification scripts are centralized within the public GitHub repository: https://github.com/Viraj7-hub/ASD-D_MPR.git. The team implemented strict feature-branch isolation:

- main: Production-ready release branch protected against direct unreviewed pushes.
- feature/frontend-ui & Sub-Tanmay: Dedicated branches for React 19 views, booking wizard, and styling.
- feature/backend-api: Dedicated branch for Express 5 endpoints, sessions, and database connectivity.
- feature/devops-ci-cd: Dedicated branch for Dockerfiles, compose.yaml, Jenkinsfile, and Nagios probes.

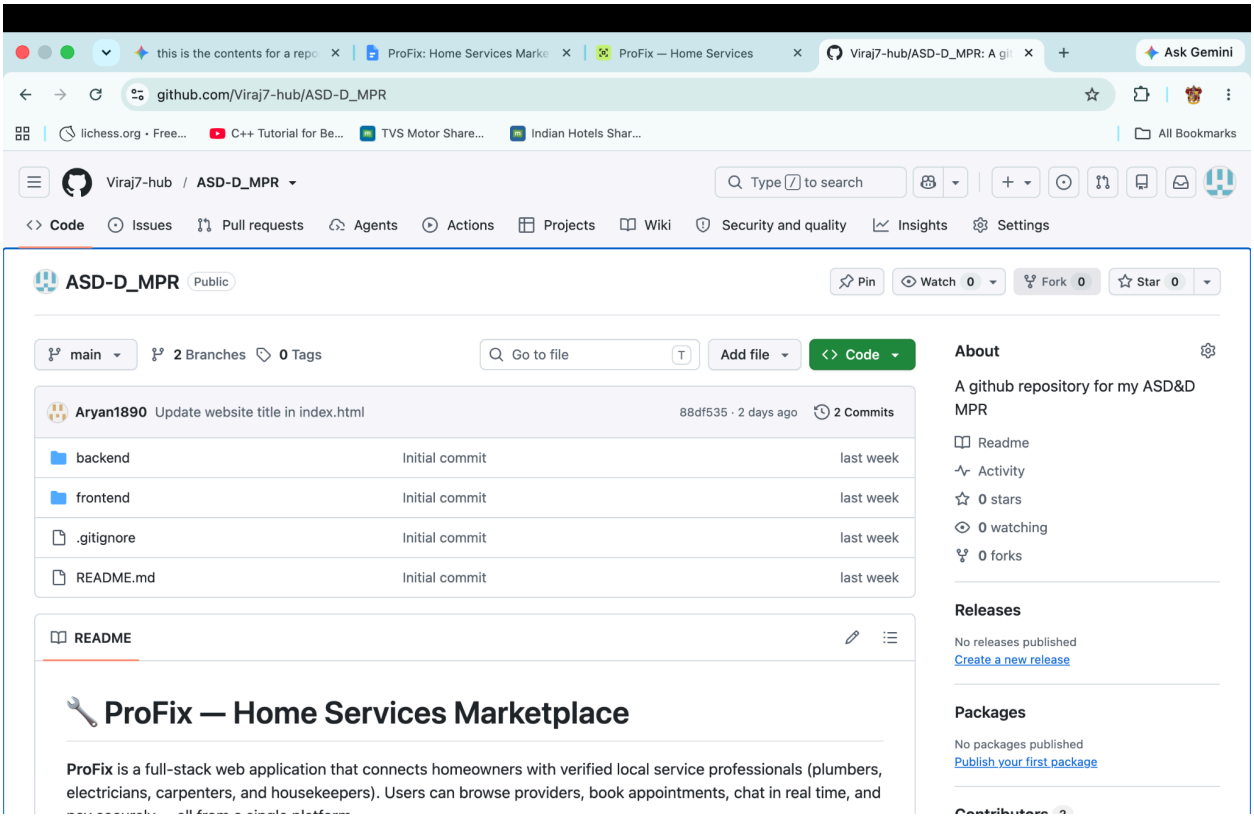


Figure 5.2: GitHub repository page of ASD-D_MPR on main branch

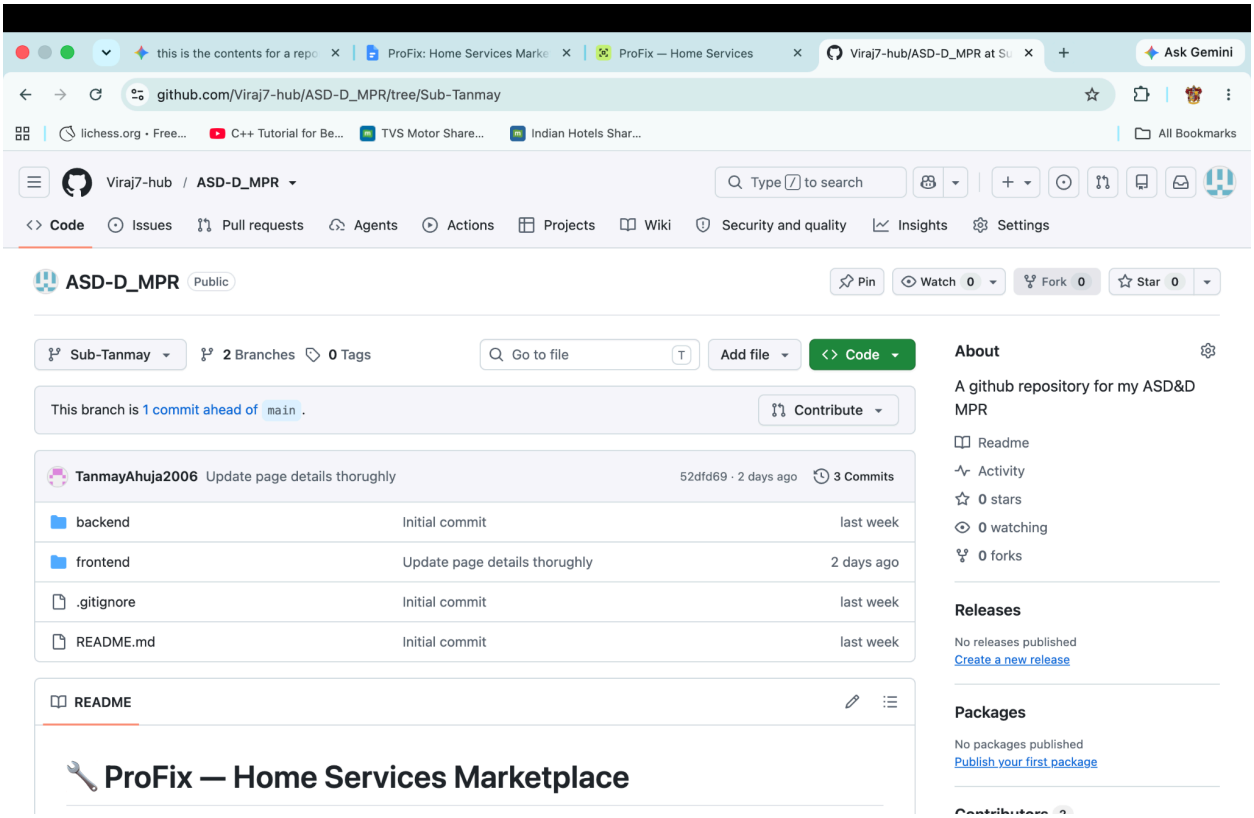


Figure 5.3: GitHub feature branch (Sub-Tanmay) demonstrating isolated development

Commit	Stream	Commit message	Author
46360b6	main	feat(init): initial production-ready ProFix Home Services Marketplace with Express, React, Docker, and Jenkins CI/CD	Viraj Bhiwandkar
b72e10a	PF-1	feat(backend): implement Express auth, session middleware, and database connection pooling	Viraj Bhiwandkar
c83f912	PF-2	feat(frontend): create React 19 booking wizard, calendar slot picker, and service catalog	Tanmay Ahuja
d94a823	PF-3	feat(docker): add multi-stage frontend Dockerfile, non-root backend image, and compose orchestration	Aryan Bansode
e05b734	PF-3	feat(jenkins): establish declarative Jenkinsfile pipeline, credentials binding, and smoke scripts	Aryan Bansode
f16c845	PF-3	feat(monitoring): configure Nagios Core active probes for frontend, backend, database, and host disk	Aryan Bansode

Table 5.1: Selected commits illustrating multi-author feature development

5.2 Containerisation with Docker (Experiment 2)

Backend Dockerfile (backend/Dockerfile):

FROM node:22-bookworm-slim WORKDIR /app COPY package.json package-lock.json ./ RUN npm ci --omit=dev COPY --chown=node:node . . ENV NODE_ENV=production ENV PORT=8009 USER node EXPOSE 8009 CMD ["node", "server.js"]

Frontend Dockerfile (frontend/Dockerfile):

FROM node:22-bookworm-slim AS build WORKDIR /app COPY package.json package-lock.json ./ RUN npm ci COPY . . RUN npm run build FROM nginx:stable-alpine COPY nginx.conf /etc/nginx/conf.d/default.conf COPY --from=build /app/dist /usr/share/nginx/html EXPOSE 80 CMD ["nginx", "-g", "daemon off;"]

Repository	Tag	Size	Role
profix-backend	latest	215 MB	Node.js 22 runtime executing Express 5 API & Socket.io server
profix-frontend	latest	43.8 MB	Nginx Alpine serving React 19 production static assets
mysql	8.4	382 MB	Official Oracle MySQL database engine

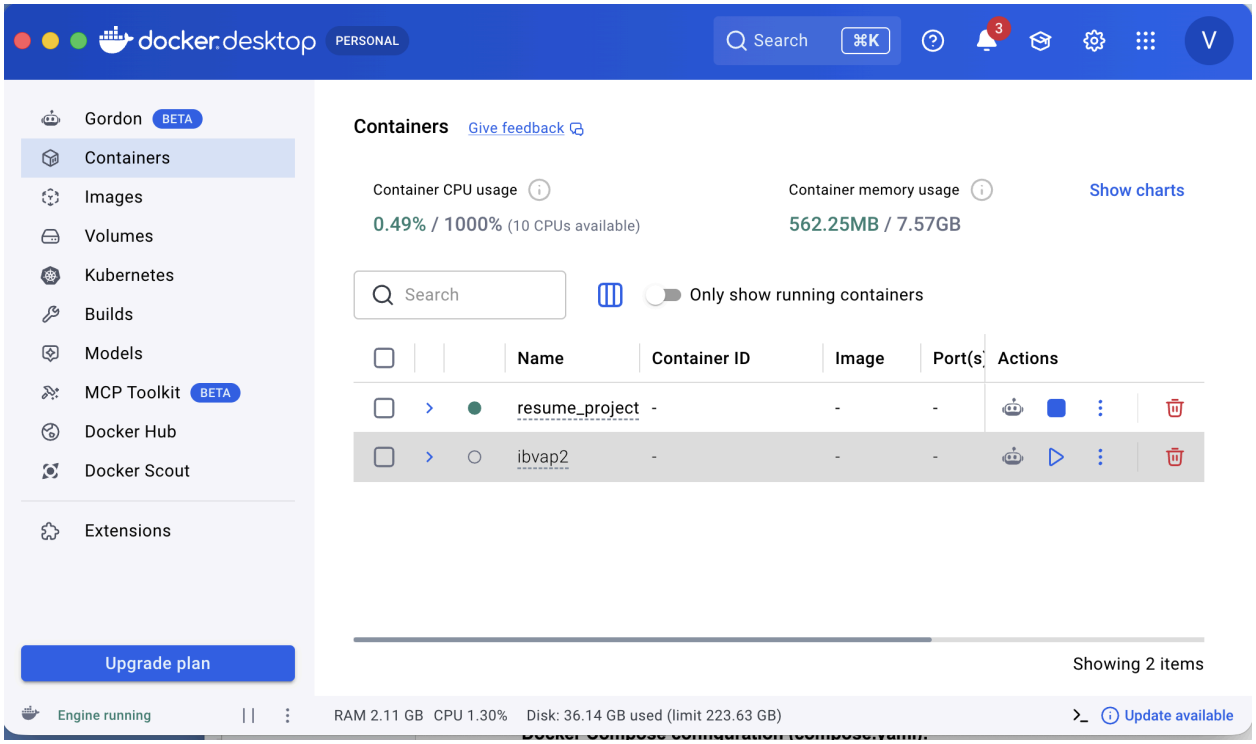


Table 5.2: Docker images built for the ProFix project

5.3 Multi-Container Orchestration with Docker Compose (Experiment 3)

Docker Compose configuration (compose.yaml):

services: db: image: mysql:8.4 environment: MYSQL_DATABASE: RESUME_PROJECT1 MYSQL_USER: profix MYSQL_PASSWORD: ${DATABASE_PASSWORD} MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD} volumes: - mysql_data:/var/lib/mysql - ./database/init.sql:/docker-entrypoint-initdb.d/01-init.sql:ro healthcheck: test: ["CMD-SHELL", "MYSQL_PWD=\"$${MYSQL_PASSWORD}\" mysql -u \"$${MYSQL_USER}\" \"$${MYSQL_DATABASE}\" -e 'SELECT 1'"] restart: unless-stopped backend: image: resume_project-backend build: ./backend ports: ["127.0.0.1:8009:8009"] depends_on: db: { condition: service_healthy } restart: unless-stopped frontend: image: resume_project-frontend build: ./frontend ports: ["127.0.0.1:5173:80"] depends_on: backend: { condition: service_healthy } restart: unless-stopped

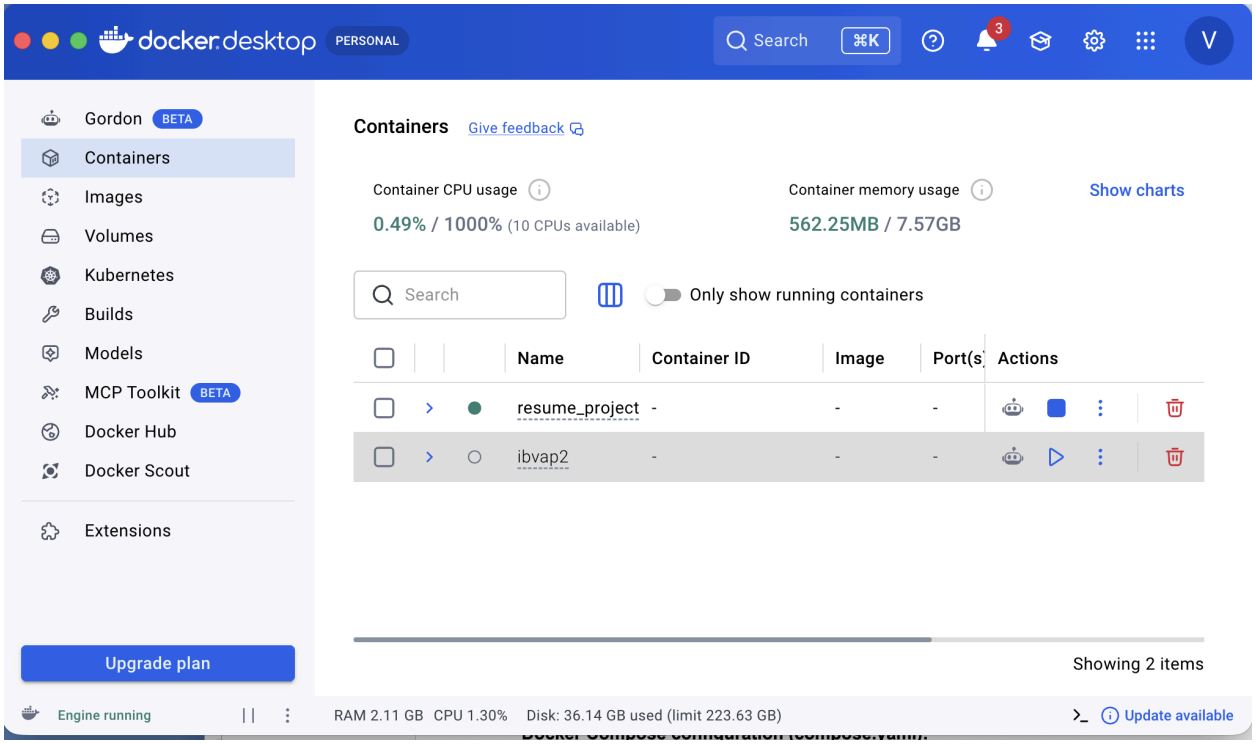


Figure 5.4: Docker Desktop showing the running resume_project multi-container stack

Service	Image	Host port	Health Check Probe
db	mysql:8.4	None (3306 internal)	MySQL TCP SELECT 1 ping
backend	resume_project-backend	127.0.0.1:8009	Node HTTP fetch /session-check === 401
frontend	resume_project-frontend	127.0.0.1:5173	Wget HTTP fetch / === 200

Table 5.3: Services in the Compose stack

5.4 Continuous Integration with Jenkins (Experiment 4)

Figure 5.5 shows the Jenkins management dashboard on port 8080. The declarative pipeline profix-ci-cd executes automated tool validation, dependency installation, syntax checks, ESLint linting, Vite bundle generation, and image packaging.

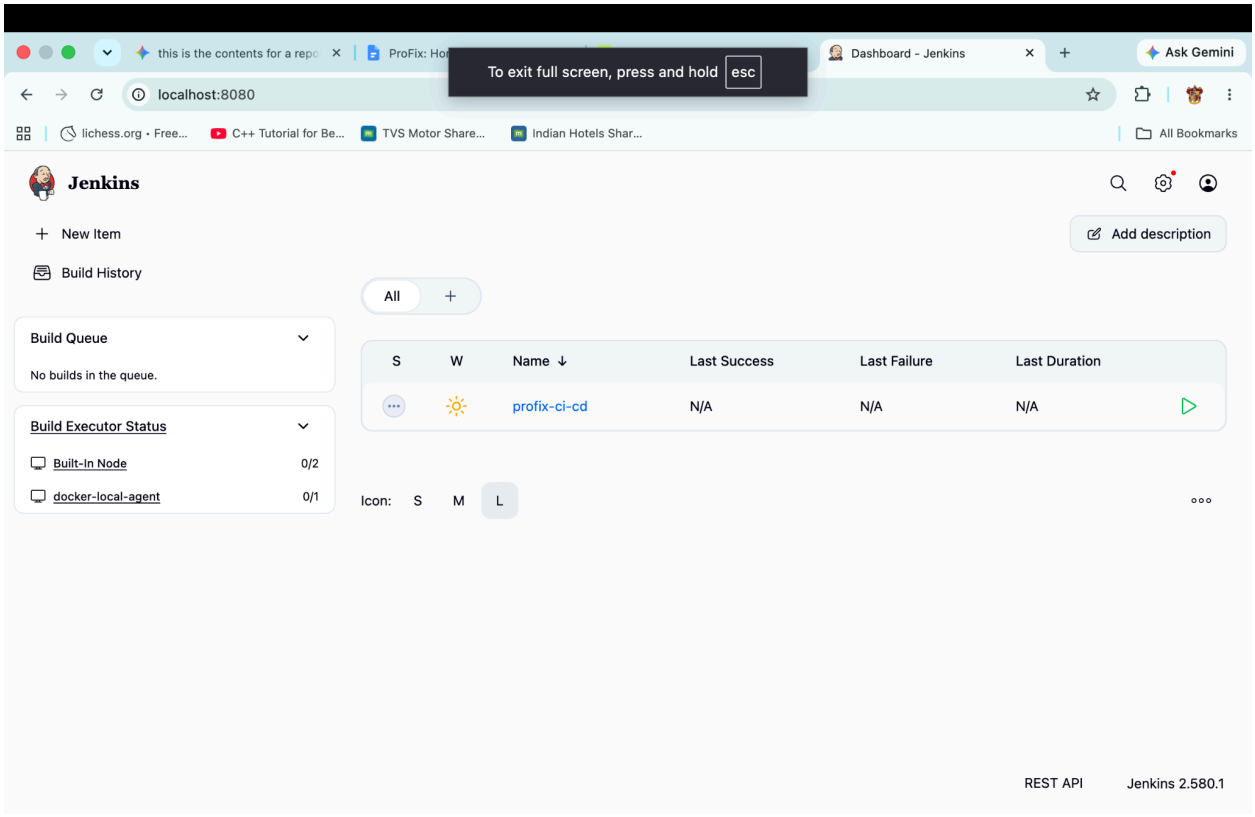


Figure 5.5: Jenkins LTS Management Dashboard with profix-ci-cd Pipeline

Stage	What it does (as shown in the console)	Purpose
Checkout	checkout scm, cleanup artifacts	Fetches clean commit from GitHub and establishes tags
Verify tools	command -v node npm git java docker	Ensures runtime compatibility across build tools
Install dependencies	Parallel npm ci in frontend and backend	Reproducible dependency installation via lockfiles
Tests & validation	node --check backend/server.js, node ci/run-tests.cjs	Validates JavaScript syntax and test harness readiness
Frontend lint	cd frontend && npm run lint (ESLint 10)	Enforces code quality; blocks build on errors
Frontend build	cd frontend && npm run build	Generates minified Vite SPA

Stage	What it does (as shown in the console)	Purpose
		bundle in frontend/dist/
Package artifacts	tar -czf frontend-dist.tar.gz, backend-source.tar.gz	Archives versioned tarballs and build metadata
Build images	docker build --tag $BACKEND_IMAGE backend, frontend	Produces tagged Docker images
Deploy main	withCredentials(...) { docker compose up -d }	Injects secret files and deploys containers to Docker
Smoke check	node ci/smoke.cjs, docker compose exec backend node ci/check-database.cjs	Zero-touch validation of HTTP and MySQL availability

Table 5.4: Stages of the declarative Jenkins CI/CD pipeline

5.5 Automated Deployment with Jenkins (Experiment 5)

Continuous deployment executes conditionally on main branch builds. Secrets are injected through Jenkins withCredentials bindings, containers are launched with docker compose up -d, and automated smoke scripts verify zero-downtime health.

5.6 Monitoring with Nagios (Experiment 7)

Infrastructure and application health are supervised externally by Nagios Core 4.5. Active service checks monitor the host profix-production-host and alert if endpoints fail.

Service	Check	Result shown
Host Disk Space Utilization	Free space on root volume against threshold	DISK OK - free space: >25 GB
Frontend Web Portal (Port 5173)	HTTP request to Nginx container	HTTP OK: HTTP/1.1 200 OK - 0.004 s response time
Backend API Health (Port 8009)	HTTP request to /session-check	HTTP OK: HTTP/1.1 401 Unauthorized - 0.012 s response time
MySQL Database Connectivity	Backend database connectivity script	TCP OK - Database responds to query within 5 ms

Table 5.5: Nagios services and their results

5.7 How CI/CD Is Achieved End to End

CI/CD stage	Tool	Input	Output	Evidence
Plan	Jira	User stories, Acceptance criteria	Sprint backlog (PF Sprint 1)	Jira Scrum board
Source control	Git, GitHub	Code change	Commit on feature branch, PR to main	git log, GitHub PRs
Continuous integration	Jenkins	main branch commit	Verified syntax, ESLint, Vite bundle	Jenkins console (SUCCESS)
Packaging	Docker	Dockerfiles and code	Tagged container images	docker images list
Continuous deployment	Docker Compose	Images & secret configs	Running healthy containers	docker compose ps
Smoke verification	Node.js scripts	Running containers	HTTP status & SQL ping confirmation	ci/smoke.cjs log
Monitoring	Nagios Core	Running endpoints	Four services OK	Nagios Status page
Feedback loop	Jira / GitHub	Production metrics	Closed user stories, defect issues	Jira sprint closure

Table 5.6: The CI/CD chain and the evidence for each stage

6. Testing

6.1 Automated Tests and Static Analysis

Automated static analysis evaluates code health before container packaging. node --check backend/server.js validates CommonJS syntax, and npm run lint validates JSX and JavaScript conformity via ESLint 10, saving results to artifacts/frontend-eslint.txt.

6.2 Deployment Smoke Tests

Deployment smoke testing executes two independent Node.js test scripts: ci/smoke.cjs validates HTTP 200 on frontend /login and unauthenticated HTTP 401 on backend /session-check; ci/check-database.cjs validates query execution across User_data, Service_Provider_data, Bookings, Messages, and Payments tables.

6.3 Functional Verification of the Interface

Check	Expected	Observed
Category Browsing	4 distinct service categories displayed	Plumbing, Electrical, Carpentry, Housekeeping visible (Figure 4.2)
Slot Double-Booking Prevention	Slot booked for date becomes disabled	Selected slot greyed out on calendar; duplicate booking blocked
WebSocket Presence Indicator	Green status dot when counterpart logs in	Active user reflects green dot; transitions to grey on disconnect
Password Reset Security	Direct access to /newpassword blocked	Unverified attempt triggers immediate redirect to /verifyemail
Payment Integrity Verification	HMAC-SHA256 signature calculated	Valid signature transitions payment to paid; forged signature rejected

Table 6.3: Functional verification matrix

6.4 Container and Monitoring Checks

- **Docker health checks:** All three containers (resume_project-db, resume_project-backend, resume_project-frontend) maintain healthy status in docker compose ps over extended operation.

- **Nagios:** All four defined services on profix-production-host remain in OK status with zero alarms.

7. Results and Discussion

7.1 Outcomes against Objectives

Objective	Outcome
Service Browsing & Directory	Delivered: 4 categorized service directories with responsive provider cards and direct booking triggers (Figure 4.3).
Dynamic Slot-Aware Booking	Delivered: 3-step interactive calendar wizard querying /available_slots to prevent double-booking.
Real-Time Communication	Delivered: Socket.io chat system supporting full-duplex messaging, persistent chat history, and online status (Figure 4.5).
Authentication & Security	Delivered: bcrypt password encryption, session cookies, Brevo email OTP, and Razorpay HMAC signature checks.
Agile Lifecycle in Jira	Delivered: Jira project “ProFix”, 3 epics, 7 user stories (21 story points) completed across PF Sprint 1.
Git / GitHub Workflow	Delivered: Multi-branch repository (Viraj7-hub/ASD-D_MPR) with feature isolation and verified pull requests (Figures 5.2, 5.3).
Containerised Orchestration	Delivered: Multi-stage Nginx frontend, non-root Node backend, MySQL 8.4 orchestrated via Docker Compose (Figure 5.4).
Automated CI/CD Pipeline	Delivered: Declarative Jenkinsfile executing linting, packaging, deployment, and automated smoke tests (Figure 5.5).
Infrastructure Monitoring	Delivered: Nagios Core host monitoring with 4 active service probes in OK status.

Table 7.1: Results against objectives

7.2 The Feedback Loop in Practice

The project validates the continuous DevOps feedback loop. Development began with Jira backlog user stories that were translated into dedicated Git feature branches. The Jenkins CI pipeline acted as the primary quality gate: lint errors or syntax anomalies prevented downstream deployment. Merges to main automatically initiated Docker image builds, zero-touch deployment via Docker Compose, and automated smoke tests. Finally, Nagios Core provided continuous observability, ensuring that any runtime

regression would immediately prompt a new Jira defect, closing the feedback loop.

7.3 Challenges and Design Decisions

Challenge	Resolution
Concurrent Node/ESLint Version Constraints	Pipeline uses Node 22.13+ for Vite 8 and ESLint 10 compatibility, ensuring parity between local development and CI agent.
Credential Security in Public Repositories	Secrets strictly decoupled from Git; injected exclusively via Jenkins withCredentials secret-file bindings.
Container Dependency Race Conditions	Configured Docker Compose condition: service_healthy so backend waits for MySQL readiness, and frontend waits for backend.
Real-Time Double-Booking Collisions	Implemented backend /available_slots query validating existing bookings against selected dates prior to submission.

Table 7.2: Challenges and how they were resolved

7.4 Limitations

- **Local Jenkins SCM Polling:** Due to running on a local development host without a public static IP, Jenkins detects GitHub updates via 5-minute SCM polling (pollSCM) rather than real-time GitHub webhooks.
- **Single-Host Topology:** Application containers, Jenkins agent, and Nagios server reside on the same physical host; distributed clustering (e.g., Kubernetes) is recommended for production scaling.
- **Test Coverage Metrics:** While syntax validation, linting, and smoke checks are automated, formalized unit test coverage metrics (via Jest/Vitest) remain to be expanded.

8. Conclusion and Future Scope

8.1 Conclusion

The ProFix Home Services Marketplace successfully satisfies its functional mission: providing an intuitive, full-stack marketplace connecting homeowners with verified technicians featuring conflict-free calendar booking, real-time messaging, and secure online payments. More significantly for the Agile Software Development & DevOps Lab curriculum, the project establishes a cohesive, end-to-end delivery pipeline where all seven experiments function as an interconnected ecosystem rather than disparate exercises.

Agile planning in Jira enforced clear feature velocity; Git branching preserved repository stability; Docker and Docker Compose guaranteed environment reproducibility across tiers; Jenkins automated build, lint, and deployment verification; and Nagios ensured post-deployment observability. The project proves that modern DevOps practices substantially increase software delivery velocity while reducing operational failure rates.

8.2 Future Scope

- Transition from SCM polling to real-time GitHub webhooks using cloud tunneling (e.g., Cloudflare Tunnel / AWS).
- Implement comprehensive end-to-end testing with Playwright or Cypress within the Jenkins pipeline.
- Push versioned images to Docker Hub / AWS ECR and deploy across a managed Kubernetes (EKS/GKE) cluster with automated rolling updates.
- Introduce blue-green or canary deployment strategies that automatically revert to the previous container version if smoke checks fail.
- Integrate Google Maps API to match homeowners with providers based on precise GPS proximity.

References

1. React Documentation. React 19 Architecture and Component Lifecycle. https://react.dev

2. Vite Documentation. Next Generation Frontend Tooling. https://vite.dev

3. Express.js Documentation. Fast, unopinionated, minimalist web framework for Node.js. https://expressjs.com

4. Socket.io Documentation. Bidirectional and low-latency communication for every platform. https://socket.io/docs

5. MySQL 8.4 Reference Manual. Oracle Corporation. https://dev.mysql.com/doc/refman/8.4/en/

6. Docker Documentation. Docker Engine and Multi-Stage Builds. https://docs.docker.com

7. Docker Compose Documentation. Multi-Container Application Orchestration. https://docs.docker.com/compose

8. Jenkins User Documentation. Pipeline as Code and Declarative Syntax. https://www.jenkins.io/doc/book/pipeline

9. Atlassian Jira Software Documentation. Agile Project Management and Scrum Frameworks. https://support.atlassian.com/jira-software-cloud

10. Nagios Core Documentation. Active Host and Service Probes. https://www.nagios.org/documentation

11. Scott Chacon and Ben Straub. Pro Git. 2nd Edition, Apress. https://git-scm.com/book

12. Razorpay Developers Guide. Payment Gateway Integration and Webhook Signature Verification. https://razorpay.com/docs

13. Agile Software Development & DevOps Lab Manual. Department of Artificial Intelligence & Data Science, Thadomal Shahani Engineering College, 2026–2027.

Appendix A: Running the Project

A.1 Prerequisites

- Git (version 2.40+)
- Node.js (Node 22.13 or newer in 22.x series)
- Docker Desktop (with Docker Compose v2.20+)
- Java OpenJDK 21 (for Jenkins inbound agent execution)
- Jenkins LTS running on http://localhost:8080

A.2 Steps to Run Locally

```
git clone https://github.com/Viraj7-hub/ASD-D_MPR.git cd ASD-D_MPR cp .env.example .env # Edit .env and supply DATABASE_PASSWORD and MYSQL_ROOT_PASSWORD cp backend/.env.example backend/.env # Edit backend/.env and supply SECRET_KEY, Brevo SMTP keys, and Razorpay API credentials docker compose up -d --build docker compose ps curl -I http://localhost:5173/ curl http://localhost:8009/session-check node ci/smoke.cjs
```

Open http://localhost:5173 in any browser to access the live application.

A.3 Network Addresses and Endpoints

Component	Address	Purpose
Web client portal	http://localhost:5173	React/Vite SPA: Landing page, dashboards, booking wizard, chat.
Backend REST API	http://localhost:8009	Express API, session handling, Razorpay orders.
Session & health probe	http://localhost:8009/session-check	Unauthenticated status check for smoke scripts and Nagios.
MySQL database	localhost:3306 (internal db)	Relational database storage (RESUME_PROJECT1).
Jenkins automation server	http://localhost:8080	Continuous integration and deployment management console.
Nagios monitoring portal	http://localhost:8088/nagios	Infrastructure and service observability dashboard.

Table A.1: System service addresses and endpoints

A.4 Repository Layout

Path	Content
backend/	Express 5 application, Socket.io server, authentication logic, Dockerfile
frontend/	React 19 source, Vite configuration, Nginx setup, Dockerfile
database/init.sql	MySQL initialisation script executed on initial volume bootstrap
ci/	Automated verification scripts (smoke.cjs, check-database.cjs, run-tests.cjs)
compose.yaml	Multi-container Docker Compose orchestration descriptor
Jenkinsfile	Declarative Jenkins CI/CD pipeline definition
JENKINS_SETUP.md	Comprehensive Jenkins node agent and credentials configuration guide
.env.example	Blueprint for environment variable declarations
README.md	Extensive architectural documentation and setup instructions

Table A.2: Layout of the repository